

Chapter 9. Feature-Based and Counting-Based Recommendations

Consider this oversimplified problem: given a bunch of new users, predict which will like our new mega-ultra-fancy-fun-item-of-novelty, or MUFFIN for short. You may start by asking which old users like MUFFIN; do those users have any aspects in common? If so, you could build a model that predicts MUFFIN affinity from those correlated user features.

Alternatively, you could ask, “What are other items people buy with MUFFIN?” If you find that others frequently also ask for JAM (just-awesome-merch), then MUFFIN may be a good suggestion for those who already have JAM. This would be using the co-occurrence of MUFFIN and JAM as a predictor. Similarly, if your friend comes along with tastes similar to yours—you both like SCONE, JAM, BISCUIT, and TEA—but your friend hasn’t yet had the MUFFIN, if you like MUFFIN, it’s probably a good choice for your friend too. This is using the co-occurrence of items between you and your friend.

These item relationship features will form our first ranking methods in this chapter; so grab a tasty snack and let’s dig in.

Bilinear Factor Models (Metric Learning)

As per the usual idioms about running in front of horses and walking after the cart, let’s start our journey into ranking systems with what can be considered the *naïve* ML approaches. Via these approaches, we will start to get a sense of where the rub lies in building recommendation systems and why some of the forthcoming efforts are necessary at all.

Let's begin again with our basic premise of recommendation problems: to estimate ratings of item $\mathbf{x}$ by user i written as $r_{i,\mathbf{x}}$. *Note the slight change in notation from earlier for reasons that will become clear momentarily.* In a usual ML paradigm, we might claim that estimating this score is done via properties of the item and the user, and frequently those properties would be described as features, and thus $\mathbf{i}$ and $\mathbf{x}$ can be the user and item vectors, respectively, composed of these features.

Now, we consider user i with their collection of previously interacted-with items $\mathcal{R}_i$, and consider $\mathcal{I} = \{\mathbf{x} | \mathbf{x} \in \mathcal{R}_i\}$ the set of vectors associated to those items in this feature space. We can then map this collection of vectors to a representation to yield a *content-based feature vector for i* .

[Figure 9-1](#) illustrates an example mapping.

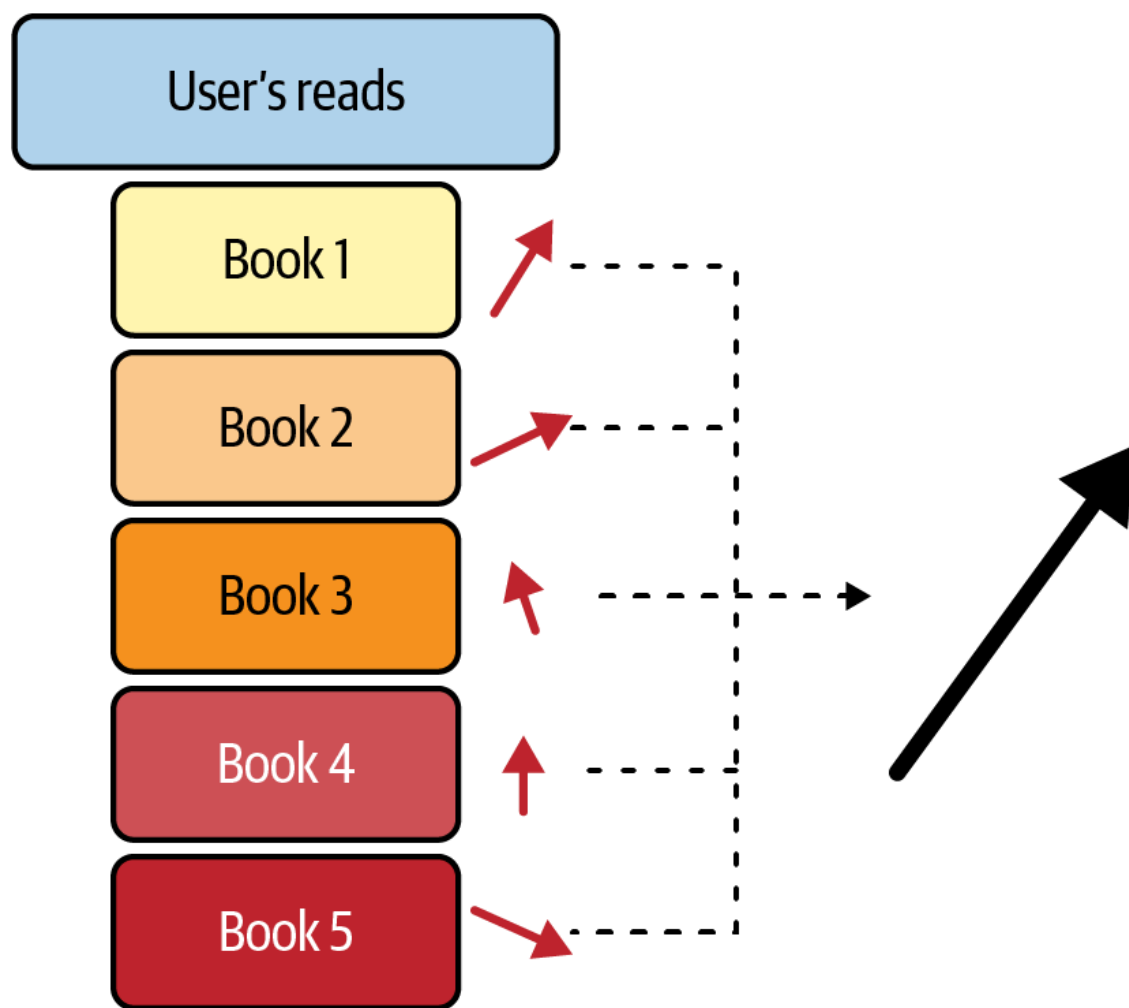


Figure 9-1. Content-to-feature vector

This extremely simple approach can turn a collection of item features and user-item interactions into features of the user. Much of the following will be increasingly rich ways of doing this. Thinking very hard about the

map, the features, and the requirements for *interaction* yields many of the key insights in the rest of the book.

Let's take the preceding mapping, $\mathbf{i} := F(\mathcal{I})$, to be a simple aggregation like dimension-wise average. Then recognize that the mapping will provide a vector of the same dimension as the items. Now we have a user vector in the same “space” as the items, and we can ask a similarity question as we did in our discussion of latent space in [Chapter 3](#).

We need to move back to the mathematical framings to set up how to use these vectors. Ultimately, we're now in a latent space with users and items, but how can we do anything with that? Well you may already remember how to compare vector similarity. Let's define the similarity to be *cosine-similarity*:

$$\text{sim}(\mathbf{i}, \mathbf{x}) = \frac{\mathbf{i} \cdot \mathbf{x}}{|\mathbf{i}| |\mathbf{x}|}$$

If we precompose our similarity with vector normalization, this is simply the inner product—and *this is an essential first step toward recommendation systems*. For convenience, let's always assume this space we're working in is after normalization, so all similarity measures are done on the unit sphere:

$$r_{i,x} \sim \text{sim}(\mathbf{i}, \mathbf{x}) = \sum_k \mathbf{i}_k \cdot \mathbf{x}_k$$

This now approximates our ratings. But wait, dear reader, where are the learnable parameters? Let's go ahead and make this a weighted summation, via a diagonal matrix $\mathbf{A}$:

$$r_{i,x} \sim \text{sim}^{\mathbf{A}}(\mathbf{i}, \mathbf{x}) = \sum_k a_k \mathbf{i}_k \cdot \mathbf{x}_k$$

This slight generalization already puts us in the world of statistical learning. You can probably already see how $\mathbf{A}$ can be used to learn which of the dimensions in this space are most important for approximating the ratings, but before we make that precise, let's generalize yet once more:

$$r_{i,x} \sim \text{sim}^{\mathbf{A}}(\mathbf{i}, \mathbf{x}) = \sum_{k,l} a_{kl} \mathbf{i}_k \cdot \mathbf{x}_l$$

This nets us even more parameters! We see that now $\text{sim}^A(\mathbf{i}, \mathbf{x}) = \mathbf{i}A\mathbf{x}$, and we are only one step away from the familiar ground of linear regression. Currently, our model is in the form of a *bilinear regression*, so let's utilize a little linear algebra. For the sake of exposition, let $\mathbf{i} \in \mathbb{R}^n$, $\mathbf{x} \in \mathbb{R}^m$, and $A \in \mathbb{R}^{n \times m}$, and then we have this:

$$\text{vect } \mathbf{i} * \mathbf{x}^T) \in \mathbb{R}^{n * m}$$

We can simplify to the following:

$$\text{sim}^A(\mathbf{i}, \mathbf{x}) = \mathbf{i}A\mathbf{x} = \text{vect } \mathbf{i} * \mathbf{x}^T) * \text{vect } A)$$

If we make up notation for the right-hand side, you'll find your friend linear regression waiting for you:

$$\mathbf{v}_{ix} := \text{vect } \mathbf{i} * \mathbf{x}^T), \beta := \text{vect } A)$$

Thus:

$$r_{i,x} \sim \text{sim}^A(\mathbf{i}, \mathbf{x}) = \mathbf{v}_{ix}\beta$$

With this computation behind us, we see that whether we wish to compute binary ratings, ordinal ratings, or likelihood estimation, the tools in our linear models toolbox can enter the party. We have available to us regularization and optimizers and any other fun we're interested in from the linear models world.

If these equations feel frustrating or painful, let me try to offer you a geometric mental model. Each item and user is in a high-dimensional space, and ultimately we're trying to figure out which ones are closest to one another. People frequently misunderstand these geometries by imagining the tips of the vectors being near one another; this is not the case. These spaces are extremely high-dimensional, which results in the analogy being far from the truth. Instead, ask if *the values are similarly large in some of the vector indices*. This is a much simpler, but also more accurate, geometric view: there are some subspaces in the extremely high-dimensional space where the vectors point in the same direction.

This forms the foundation for where we are going but has serious limitations for large-scale recommender problems. You will see, however, that

the feature-based learning still has its place in the cold-start regime.

Note that in addition to the preceding approach of building content-based features for a user, we may also have obvious user features that are obtained via queries to the user, or implicitly via other data collection; examples of these features include location, age range, and height.

In this section, we've discussed ways to put users and items in the same latent spaces. We claim that we can make comparisons between users and items by vector operations. In mathematics, vectors are elements of vector spaces, and (finite dimensional) vector spaces are defined by their number of dimensions and the values that the vectors have as elements. For example, if we say it's a three-dimensional vector space with 8-bit integers, that's sufficient to specify a vector space.

However, devilish details are lurking around. First, what does *distance* mean in a specified vector space? We have many conventional measures, but it's important to ensure that comparisons between two spaces are utilizing the same definitions of distance. Another consideration is the process by which you define the vectors of the space; if you arrive at your vectors via a dimension reduction from a larger space, there are likely density properties that you can expect not to be present naively. Where this is most relevant for ranking models and recommendation systems is that we frequently arrive at user space and item space separately, and often compute distance between user and item vectors.

Is this OK? In many cases, it lacks firm theoretical footing but works well. One particular case where this *does* have a firm theoretical footing is MF. Rather than a long digression on geometric algebra, we will give the following guidance: if you're interested in comparing two vectors that aren't in the same space, ask yourself if they're of the same dimension, if distance is defined the same in both spaces, and if the density priors are similar. In fact, at times, none of these is true and you can *still* get away with a comparison. But for each of these potential risks, it's worth a stop-and-think.

One explicit example of a troubling difference in two latent spaces is found in [“Poincaré Embeddings for Learning Hierarchical Representations”](#) by Maximilian Nickel and Douwe Kiela; this paper provides an interesting way to encode relationships between items in your latent space via the implicit geometry. However, your users may not be

encoded in a hyperbolic space. Tread carefully if you compute inner products between these and Euclidean embedded vectors!

Feature-Based Warm Starting

As you saw in [Chapter 7](#), there are a variety of ways to use features alongside some of the collaborative filtering (CF) and MF approaches we've presented. In particular, you saw how encoders built via a two-towers architecture can be used for fast feature-based recommendations in the cold-start scenario. Let's look into this deeper and think carefully about features for new users or items.

In [Chapter 9](#), we built our bilinear factor model as a simple regression and, in fact, saw that all the standard ML modeling approaches would apply. However, we took the user embedding to be features learned from item interactions: that is, the content-based feature vector. If our goal is to build a recommendation algorithm that does not need a history of user ratings, obviously this construction will not suffice.

We might begin by asking if the preceding factor regression approach could work in the pure user-feature setting—leave aside worries about the inner product that depended on a mutual embedding and just take everything to be pure matrices. While this is a reasonable idea that can yield some results, we may quickly identify the coarseness of this model: each user would then need to provide answers to queries $\mathbf{q}_k$ such that $\mathbf{i} \in \mathbb{R}^k$. Because the dimensionality of these user vectors scales linearly with the number of questions we're willing and able to ask the user, we are passing along the difficulty of the problem to our user experience.

Because we intend on using CF via MF as our core model, we'd really like to find a way to smoothly transition from the feature-based model into this MF, ensuring we take advantage of user/item ratings as they emerge. In [“The Evaluation Flywheel”](#), we discussed using inference results and their subsequent outcomes in real time to update the model, but how do we account for that in the modeling paradigm?

In a latent-factor model obtained via MF, we have the following:

$$\mathbf{u}_i \mathbf{v}_x$$

Here, $\mathbf{u}_i$ has a Gaussian prior with zero mean; this is why new users won't yield useful ratings before they have interaction data. We thus say that the *user-matrix* has *zero-concentrated priors*. Our first strategy to including features in our MF is to simply build a better priors distribution.

More mathematically: we learn a regression model $G(\mathbf{i}) \sim \mathbf{u}_i$ for initialization of our learned factor matrix, and this means we're learning the following:

$$s(i, x) \sim \mathbf{w}_{ix}\gamma + \alpha_i + \beta_x + \mathbf{u}_i \mathbf{v}_x$$

Here, our $\mathbf{w}_{ix}\gamma$ is now a standard bilinear feature regression from user and item features, the bias terms are learned to estimate popularity or *rank inflation*, and our familiar MF terms are $\mathbf{u}_i \mathbf{v}_x$.

Note that this approach provides a general strategy for including features into an MF model. How we fit the factors-features model is totally up to us, as are the optimization methods we wish to employ.

Also note that instead of regression-based approaches, priors can be established via *k*-nearest neighbors in a purely feature-based embedding space. This modeling strategy is explored in great detail in [“Eliciting Auxiliary Information for Cold Start User Recommendation: A Survey”](#) by Nor Aniza Abdullah et al. Compare this with the item-item content-based recommender from [Chapter 5](#), where the query is an item and similarity in item space is the link between the last item and the next.

We have established a strategy and a collection of approaches to building our models via features. We've even seen how our MF will fall over for new users, only to be saved by a feature-based model. So why not stick to features? Why introduce factors at all?

Segmentation Models and Hybrids

Similar to our preceding discussion of warm-starting via features is the closely related concept of *demographic-based systems*. Note that *demo-*

graphic in this context need not refer explicitly to personally identifiable information and can refer to the user data collected during the sign-up process. Simple examples from book recommendations might include a user's favorite genres, self-identified price preference, book-length preferences, and favorite author. Standard methods of clustering-based regression can be helpful in converting a small set of user features into recommendations for new users. For these coarse user features, building simple feature-based models like naive Bayes, can be especially effective.

More generally, given user feature vectors, we can formulate a similarity measure and then user segments to make new-user recommendations. This should feel similar to feature-based recommenders, but instead of requiring usage of user features, we model the user's containment in a segment and then build our factor model from the segment to different items.

One way to imagine this approach is to consider the modeling problem as estimating the following for C , a user cluster:

$$r_{C,x} := \text{Avg}(r_{i,x} \mid i \in C)$$

Then we estimate $P(\mathbf{j} \in C)$, the probability a user $\mathbf{j}$ is a member of C . We can easily imagine that we instead wish to use the probability associated with each cluster to build a bagging model, and have each cluster contributed to a weighted average rating.

While these ideas may not seem like interesting extensions to what we've built previously, in practice they can be enormously useful for fast, explainable recommendations for new users.

Also note that nothing in this construction is particular to the users; we can consider the *dual model* that takes the clustering to be at the level of the items and performs a similar process. Combining these models can provide the coarsest model of simply user segments to item groups, and utilizing several of these modeling approaches simultaneously can provide important and flexible models.

Tag-Based Recommenders

One special case of the segmentation model for item-based recommenders is a *tag-based recommender*. This is a quite common first recommender to try when you have some human labels and need to quickly turn it into a working recommender.

Let's talk through a toy example: you have a personal digital wardrobe, where you've logged many features about each article of clothing in your personal closet. You want your fashion recommender to give you suggestions for what else to wear, given that you've selected one piece for the day. You wake up and see that it's rainy outside, so you start by choosing a cozy cardigan. The model you've trained has found that cardigan has tags *outerwear* and *cozy*, which it knows correlate well with *bottoms* and *warm*—so it's likely to recommend heavier jeans today.

The upside of a tag recommender is how explainable and understandable the recommendations are. The downside is that performance is directly tied to the amount of effort that's put into tagging items.

Let's discuss a slightly more involved example of a tag-based recommender that one of the authors built in collaboration with Ashraf Shaik and Eric Bunch for recommending blog posts.

The goal was to warm-start the blog-post recommender by utilizing high-quality tags that classified the blogs into themes. One special aspect of this system was its rich hierarchical tagging maintained by the marketing team. In particular, each *tag type* had several values, and there were 11 tag types with up to 10 values each. Blogs had values for each tag type and sometimes had multiple tags in a single tag type for the blog. This may sound a bit complicated, but suffice it to say that each blog post could have some of the 47 tags, and the tags were further grouped into types.

One of the first potential tasks is to use those tags to build a simple recommender, and we did, but doing so would mean missing a significant additional opportunity when afforded such high-quality tag data: evaluating our embeddings.

First, we needed to understand how we could build user embeddings. Our plan was to average the blog embeddings a user had seen, a simple CF approach when you have a clear item embedding. Thus we wanted to train the best embedding model possible for these blogs. We started by considering models like BERT but were unsure whether the highly technical content would be meaningfully captured by our embedding model. This led us to realize that we could use the tags as a classifier dataset for our embedding. If we could test several embedding models by training a simple multilayer perceptron (MLP) to perform multilabel multiclassification for each tag type, where the input features were the embedding dimensions, then our embedding space would capture the content well.

Some of the embedding models were of varying dimensions, and some were quite large, so we also first used a dimension reduction (UMAP) to a standard size before we trained the MLP. We used [F1 scores](#) to determine which of the embedding models led to the best classification model for tags, and we used visual inspection to ensure the groups were as we'd hoped. This worked quite well and showed that some embeddings were much better than others.

Hybridization

You saw in the previous section how to blend our MF with simpler models by taking priors from the simpler models and learning how to transition away. Coarser approaches to this process of *hybridization* exist:

Weighted combinations of models

This approach is incredibly powerful, and the weights can be learned in a standard Bayesian framework.

Multilevel modeling

This approach can include learning a model to select which recommendation model should be used, and then learning models in each regime. For example, we could use a tree-based model on user features when the user has fewer than 10 historical ratings and then use MF after that. A variety of multilevel approaches exist, in-

cluding *switching* and *cascading*, which correspond roughly to voting and boosting, respectively.

Feature augmentation

This allows multiple vectors of features to be concatenated and a larger model to be learned. By definition, if we wish to combine feature vectors with factor vectors, like those coming from a CF, we will expect substantial nullity. Learning despite that nullity allows a somewhat naive combination of the different kinds of features to be fed into the model and operated on in all regimes of user activity.

We can combine these models in a variety of useful ways. However, we take the position that instead of more complicated combinations of several models that work well in different paradigms, we will attempt to stick to a relatively straightforward model-service architecture by doing the following:

- Training the best model we can by using MF-based CF
- Using user and item feature-based models for cold start

Let's see why we think feature-based modeling might not be the best strategy, even if we do it via neural networks and latent factor models.

Limitations of Bilinear Models

We started this chapter by describing *bilinear modeling* approaches, and immediately you should take warning—they're linear relationships. You can immediately wonder, "Are there really linear relationships between the features of my users and items and the pairwise affinity?"

The answer to this question might depend on the number of features, or it might not. Either way, skepticism is appropriate, and in practice the answer is overwhelmingly *no*. You might think, "Well then, as it is a linear approximation, MF also cannot succeed," but that's not so clear-cut. In fact, MF suggests that the linear relationship is *between the latent factors*,

not the actual features. This subtle difference makes a world of difference.

One important callout before we move on to simpler ideas is that neural networks with nonlinear activation functions can be used to build feature-based methods. This domain has had some successes, but ultimately a surprising and important result is that neural CF does not outperform matrix factorization. This doesn't suggest that there are no useful approaches for feature-based models utilizing MLPs, but it does defray some of our worries about MF being *too linear*. So why not use more feature-based approaches?

The first most obvious challenge for content-based, demographic-based, and any other feature-based method is *getting the features*. Let's consider the dual problems:

Features for users

If we want to collect features for users, we need to either ask them a series of queries or infer those features implicitly. Inferring these via exogenous signals is noisy and limited, but each query that we ask the user increases the likelihood of onboarding drop-off. When we think of user-onboarding funnels, we know that each additional prompt or question incurs another chance that the user will not complete the onboarding. This effect accumulates quickly, and without users making it through the funnel, the recommendation system won't be very useful.

Features for items

On the flip side, creating features for items is a heavily manual task. While many businesses need to do this task to serve other purposes as well, it still incurs a significant cost in many cases. If the features are to be useful, they need to be of high quality, which incurs more debt. But most importantly, if the number of items is extremely large, the cost may quickly get out of reach. For large-scale recommendation problems, manually adding features is simply infeasible. This is where automatic feature-engineering models can help.

Another significant issue in these feature-based models is *separability* or *distinguishability*. These models are not useful if the features cannot separate the items or users well. This leads to compounding problems as the cardinality increases.

Finally, in many recommendation problems, we start with the assumption that taste or preference is extremely personal. We fundamentally believe that our interest in a book will have less to do with the number of pages and publication date than how it connects with us and our personal experience (*our deepest apologies to anyone who bought this book based on page number and publication date*). CF—while simple in concept—speaks better to these connections via a *shared experience network*.

Counting Recommenders

Here we will use the simplest feature type, simple counting. Counting the frequency and pairwise frequencies will provide a simple but useful set of initial models.

Return to the Most-Popular-Item Recommender

Our super simple scheme from before, implementing the MPIR, provided us with a convenient toy model, but what are the practical considerations of deploying an MPIR? It turns out that the MPIR provides an excellent framework for getting started on a Bayesian approximation approach to recommendations. Note that in this section, we're not even considering a personalized recommender; everything here is reward maximization across the entire user population. We follow the treatment in [*Statistical Methods for Recommender Systems*](#) by Deepak K. Agarwal and Bee-Chung Chen (Cambridge University Press).

For the sake of simplicity, let's consider *click-through rate (CTR)* as our simple metric to optimize. Our formulation is as follows: we have $\mathcal{I} = \{i\}$ items available to recommend and initially *only one time period* in which to do it, and we're interested in an *allocation plan*, or a set of proportions x_i , $\sum_{i \in \mathcal{I}} x_i = 1$, for how to recommend items. This can be

seen as a very simple multiarmed bandit problem with the reward given by the following:

$$R(\mathbf{x}, \mathbf{c}) = \sum_{i \in \mathcal{I}} c_i * (N * x_i)$$

Here, c_i represents prior distributions of CTR for each item. It's plain to see that maximizing this reward is achieved by allocating all recommendations to the item with greatest p_i , i.e., picking the most popular item in terms of CTR.

This setup makes it obvious that if we have strong confidence in our priors, this problem seems trivial. So let's move to a case where we have a mismatch in confidence.

Let's consider *two time periods*, N_0 and N_1 , as indicating the number of user visits. Note that we think of 0 as the past and 1 as the future in this model. Let's assume that we offer *only two items* and that, somewhat mysteriously, for one item we have 100% confidence in its CTR in each time period: q_0 and q_1 will denote these rates, respectively. In contrast, we have only priors for our second item: $p_0 \sim \mathcal{P}(\theta_0)$ and $p_1 \sim \mathcal{P}(\theta_1)$ will denote these rates, respectively, and we regard θ_i as a state vector. We again notate the allocations with $x_{i,t}$, where now the second index refers to time period. Then we can simply compute the expected number of clicks as follows:

$$\mathbb{E} [N_0 * x_0 p_0 - q_0) + N_1 * x_1 p_1 - q_1)] + q_0 N_0 + q_1 N_1$$

This is maximized by assuming a distribution for p_1 as a function of x_0 and p_0 . With distributional assumptions that p_0 is gamma distributed and p_1 is normally distributed, we can treat this as a convex optimization problem to maximize the clicks. See *Statistical Methods for Recommender Systems* for a full treatment of the statistics.

This toy example extends in both dimensions to model larger item sets and more time windows and provides us with relatively straightforward intuition about the relationship between our priors for each item and time step during this step-forward optimization.

Let's put this recommender in context: we've started with item popularity and generalized to a Bayesian recommender that learns with respect to user feedback. You might consider a recommender like this for a very trend-based recommendations context like news; popular stories are often important, but that can change rapidly, and we want to be learning from user behavior.

Correlation Mining

We've seen ways to use correlations between features of items and recommendations, but we should not forget to use correlations between items themselves. Think back to our early discussions of cheese in [Chapter 2 \(Figure 2-1\)](#); we said that our CF gave us a way to find mutual cheese tastes to recommend new cheeses. This was built on the notion of ratings, but we can abstract away from the ratings and simply look at the correlations of items a user chooses. You can imagine for an ecommerce bookseller that a user's choice of one book to read may be useful in recommending others—even if that user chooses not to rate the first book. We also saw this phenomena in [Chapter 8](#) as we used the co-occurrence of tokens in Wikipedia entries.

We introduced the co-occurrence matrix as the multidimensional array of counts where two items, i and j , co-occur. Let's take a moment to discuss co-occurrence a bit more deeply.

Co-occurrence is context dependent; for our Wikipedia articles, we considered co-occurrence of tokens in an article. In the case of ecommerce, co-occurrence can be two items purchased by the same user. For ads, co-occurrence can be two things that the user clicked, and so on.

Mathematically, given users and items, we construct an *incidence vector* for each user, the binary vector of one-hot encoded features for each item that they interacted with. Those vectors are stacked into a vector to yield a $\#(users) \times \#(items)$ matrix in which each row is a user, each column is an item, and the elements equal 1 when a user-item pair has interacted.

To be mathematically precise, a *user-item incidence structure* is a collection of sets of user interactions, $\mathbf{y}_u\}_{u \in U}$, with items $\mathbf{x}_i\}_{i \in I}$, where U indexes users and I indexes items.

The associated *user-item incidence matrix*, $\mathcal{U}$, is the binary matrix with rows indexed by sets, and columns indexed by nodes, such that elements are as follows:

$$e_{\mathbf{y}_u, \mathbf{x}_i} = \begin{cases} 1 & \mathbf{x}_i \in \mathbf{y}_u \\ 0 & \text{otherwise} \end{cases}$$

The *co-occurrence of $\mathbf{x}_a$ and $\mathbf{x}_b$* is the order of the set $\mathbf{y}_u \mid \mathbf{x}_a \in \mathbf{y}_u \text{ and } \mathbf{x}_b \in \mathbf{y}_u\}$. We can also write that as a matrix that can be computed via a simple formula; let $C_{\mathcal{J}}$ be the co-occurrences matrix—i.e., the matrix with rows and columns indexed by $\mathbf{x}_i\}_{i \in I}$ and with elements that are the co-occurrences of the indices. Then we use the following:

$$C_{\mathcal{J}} = \mathcal{J}^T * \mathcal{J}$$

HIGHER-ORDER CO-OCCURRENCES

You could imagine further generalizing this recommender to aggregate across several items the user has seen. In practice, you could consider the last five items the user has interacted with and then compute the conditional-MPIR recommendations for each and union them together.

Alternatively, you could generalize to *higher-order* co-occurrences. In other words, instead of pairs of items that co-occur, look at triples, quadruples, or more. To read one approach to this generalization, check out [“Higher Order Co-occurrence Tensors for Hypergraphs via Face-Splitting”](#) by one of the authors.

As mentioned in [“Customers Also Bought”](#), we can build a new variant of our MPIR by considering the rows or columns of the co-occurrence matrix. The *conditional MPIR* is the recommender that returns the max of the elements in the row corresponding to $\mathbf{x}_i$, given the user’s last interaction was the item $\mathbf{x}_i$.

In practice, we often think of the row corresponding to $\mathbf{x}_i$ as a *basis vector*, i.e., a vector $\mathbf{q}_{\mathbf{x}_i}$ with one nonzero element in the $\mathbf{x}_i$ th position:

$$\mathbf{q}_{\mathbf{x}_i, j} = \begin{matrix} 1 & j = \mathbf{x}_i \\ 0 & \text{otherwise} \end{matrix} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Then we can consider max—or even softmax—of the preceding dot products:

$$C_{\mathcal{J}} = \mathcal{J}^T \cdot \mathcal{J} * \mathbf{q}_{\mathbf{x}_i}$$

This yields the vector of co-occurrence counts between $\mathbf{x}_i$ and each other item. Here we frequently will call $\mathbf{q}_{\mathbf{x}_i}$ a *query* to indicate that it's the input to our co-occurrence recommendation model.

HOW DO YOU STORE THIS DATA?

We can think about co-occurrence data in a *lot* of ways. The main reason is because we expect that co-occurrences for recommendation systems are incredibly sparse. This means that the preceding method of matrix multiplication—which is approximately $O(n^3)$ —is going to be relatively slow to compute fewer nonzero entries. Because of this and concerns about storing huge matrices full of zeros, computer scientists have taken seriously the problem of representing sparse matrices.

[Max Grossman](#) claims there are 101 ways, but in practice there are only a few. JAX supports [BCOO](#), or *batched coordinate format*, which is essentially a list of coordinates for nonzero elements, and then what those elements are.

In our binary case of interactions, those are 1s, and for the co-occurrence matrix, those are the counts. The structure of these matrices can be written as follows:

```
{
  'indices': indices,
  'values': values,
  'shape': [user_dim, items_dim]
}
```

Pointwise Mutual Information via Co-occurrences

An early recommendation system for articles used *pointwise mutual information*, or PMI, which is closely related to co-occurrences. In the context of NLP, PMI attempts to express how much more frequent co-occurrence is than random chance. Given what we've seen before, you can think of this as a normalized co-occurrences model. Computational linguists frequently use PMI as an estimator for word similarity or word meaning following from the distributional hypothesis:

You shall know a word by the company it keeps.

—John R. Firth, British linguist

In the context of recommendation ranking, items with very high PMI are said to have a highly meaningful co-occurrence. This can thus be used as an estimator for *complementary* items: given you've interacted with one of them, you should interact with the other.

PMI is computed for two items, x_i, x_j , via the following:

$$\frac{p(x_i, x_j)}{p(x_i) * p(x_j)} = \frac{C_{\mathcal{I}}(x_i, x_j) * \#(\text{total interactions})}{\#(x_i) * \#(x_j)}$$

The PMI calculation allows us to modify all our work on co-occurrence to a more normalized computation, and thus is a bit more meaningful. This process is related to the GloVE model we learned in [“GloVE Model Definition”](#). The negative PMI values allow us to understand when two things are not often witnessed together.

These PMI calculations can be used to recommend *another item in a cart* when an item has been added and you find those with very high PMI. It can be used as a retrieval method by looking at the set of items a user has already interacted with and finding items that have high PMI with several of them.

Let's look at how to turn co-occurrences into other similarity measures.

A good question to consider at this point is “Is PMI between two objects a measurement of distance? Can I define similarity directly as the PMI between two items, and thus yield a convenient geometry in which to consider distances?” The answer is no. Recall that one of the axioms of a distance function is the triangle inequality; a useful exercise is to consider why the triangle inequality would not be true for PMI.

But all is not lost. In the next section, we’ll show you how to formulate some important similarity measurements from co-occurrence structures. Further, in the next chapter, we’ll discuss Wasserstein distance, which allows you to turn the co-occurrence counts into a distance metric directly. The key difference will be considering the co-occurrence counts of all other items simultaneously as a distribution.

Similarity from Co-occurrence

Earlier, we discussed similarity measures and how they come from the Pearson correlation. The Pearson correlation is a special case of similarity when we have explicit ratings, so let’s instead look at when we don’t.

Consider incidence sets associated to users, $y_u\}_{u \in U}$, as we define three distance metrics:

Jaccard similarity, $Jac(-)$

The ratio of shared items by two users to the total items those users have interacted with

Sørensen-Dice similarity, $DSC(-)$

Twice the ratio of shared items by two users to the sum of total items each user has interacted with

Cosine similarity, $Cosim(-)$

The ratio of shared items by two users to the product of total items each user has interacted with

These are all very related metrics with slightly different strengths. Here are some points to consider:

- Jaccard similarity is a real distance metric that has some nice properties for geometry; neither of the other two is.
- All three are on the interval $[0, 1]$, but you'll often see cosine extended to $[-1, 1]$ by including negative ratings.
- Cosine can accommodate “thumbs-up/thumbs-down” by merely extending all interactions to have a polarity of ± 1 .
- Cosine can accommodate “multiple interactions” if you allow the vectors to be nonbinary and count the number of times a user interacts with an item.
- Jaccard and Dice are related by the simple equation $S = 2J/(1 + J)$, and you can easily compute one from the other.

Notice that we've defined all these similarity measures between users. We'll show in the next section how to extend these definitions to items and how to turn these into recommendations.

Similarity-Based Recommendations

In each of the preceding distance metrics, we've defined a similarity measure, but we haven't yet discussed how similarity measures turn into recommendations. As we discussed in [“Nearest Neighbors”](#), we utilize similarity measures in our retrieval step; we wish to find a space in that items that are *close* to one another are good recommendations. In the context of ranking, our similarity measure can be used directly to order the recommendations in terms of how likely the recommendation is relevant. In the next chapter, we'll talk more about metrics of relevance.

In the preceding section, we looked at three similarity scores, but we need to expand our notion of the relevant sets for these measures. Let's consider Jaccard similarity as a prototype.

Given a user y_u and an unseen item x_i , let's ask, “What is the Jaccard similarity between this user and item?” Let's remember that Jaccard similarity is the similarity between two sets, and in the definition those sets were

both *incidence sets of users' interactions*. Here are three ways to use this approach for recommendations:

User-user

Using our preceding definition, find the k users with maximum Jaccard similarity. Compute the percentage of these users who have interacted with x_i . You may also wish to normalize this by popularity of the item x_i .

Item-item

Compute the set of users that each item has interacted with, and compute the k items most similar to x_i with respect to Jaccard similarity of these item-user incidence sets. Compute the percentage of these items that are in y_u 's set of interactions. You may also wish to normalize this by total interactions of y_u or the popularity of the similar items.

User-item

Compute the user y_u 's set of items they've interacted with, and the set of items co-occurring with x_i in any user's incidence set of interaction. Compute the Jaccard similarity between these two sets.

Frequently in designing ranking systems, we specify the *query*, which refers to which nearest neighbors you're looking for. We then specify how you use those neighbors to yield a recommendation. The items that may become the recommendation are the candidates, but as you saw in the preceding example, the neighbors may not be the candidates themselves. An additional complication is that you usually need to compute many candidate scores simultaneously, which requires optimized computations that we'll see in [Chapter 16](#).

Summary

In this chapter, we've begun to dig deeper into notions of similarity—building on our intuition from retrieval that users' preferences might be

captured by the interactions they've already demonstrated.

We started out with simple models based on features about users and built linear models relating them to our target outcomes. We then combined those simple models with other aspects of feature modeling and hybrid systems.

Next, we moved into discussing counting—in particular, counting the co-occurrence of items, users, or baskets. By looking at frequent co-occurrence, we can build models that capture “If you liked a , you may like b .” These models are simple to understand, but we can use these basic correlation structures to build similarity measures, and thus latent spaces where ANN-based retrieval can yield good candidates for recommendations.

One point that you may have noticed about the featurization of all the items and the building of our co-occurrence matrices is that the number of features is astronomically large—one dimension for each item! This is the area of investigation we'll tackle in the next chapter: how to reduce the dimensionality of your *latent space*.